PYCON HK 2026 Hong Kong · 14–15 November 2026

PYCON HK 2026

PYTHON × EBPF

Python eBPF BCC C-String Runtime vs Python-BPF AST Ahead of Time

BCC compiles C at runtime. **Python-BPF** compiles Python ahead of time. We measured both to 2 million events.



Nizar Akbar Meilani · DomaiNesia, Yogyakarta

slides + dataset → github.com/nizarakbarm

Day job

Systems engineer at **DomaiNesia** — hosting and cloud, Yogyakarta, Indonesia.

Focus

eBPF *from Python* — kernel observability without writing C.

Back at PyCon HK

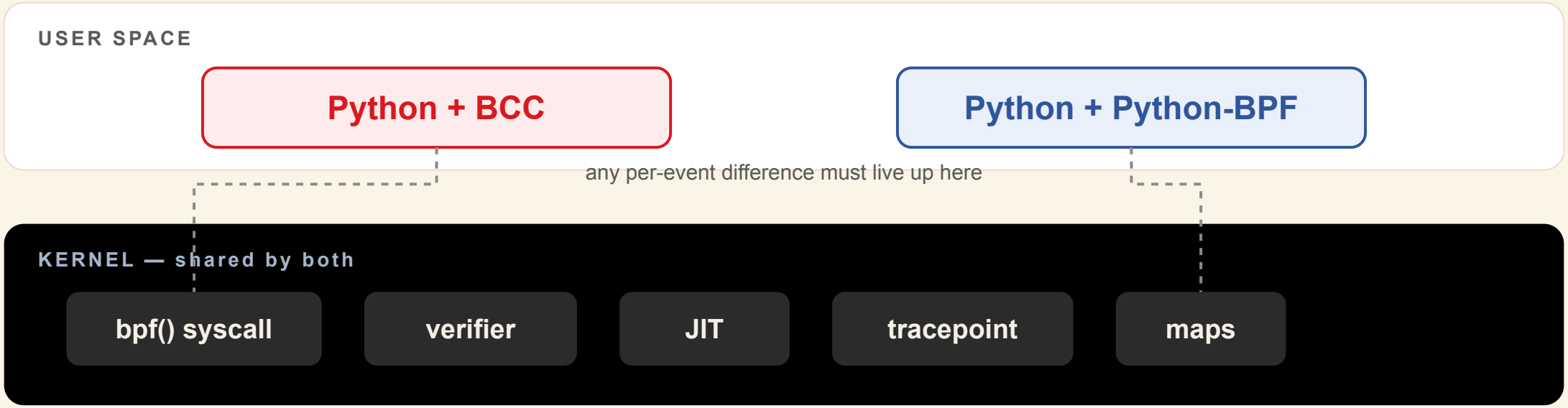
Third year running. Twice at APAC, once at PyCon ID.

This talk is one habit from CTF work: **measure every layer, trust no layer**.

TODAY | one question, four parts

01	**Which starts faster?** from process launch to a loaded BPF program	`startup`
02	**Which costs less per event?** same program, same workload, 2 million events	`per event`
03	**Who does the waiting?** the loop that reads events back into Python	`the wait`
04	**What do you lose?** speed is not the only axis	`the limits`

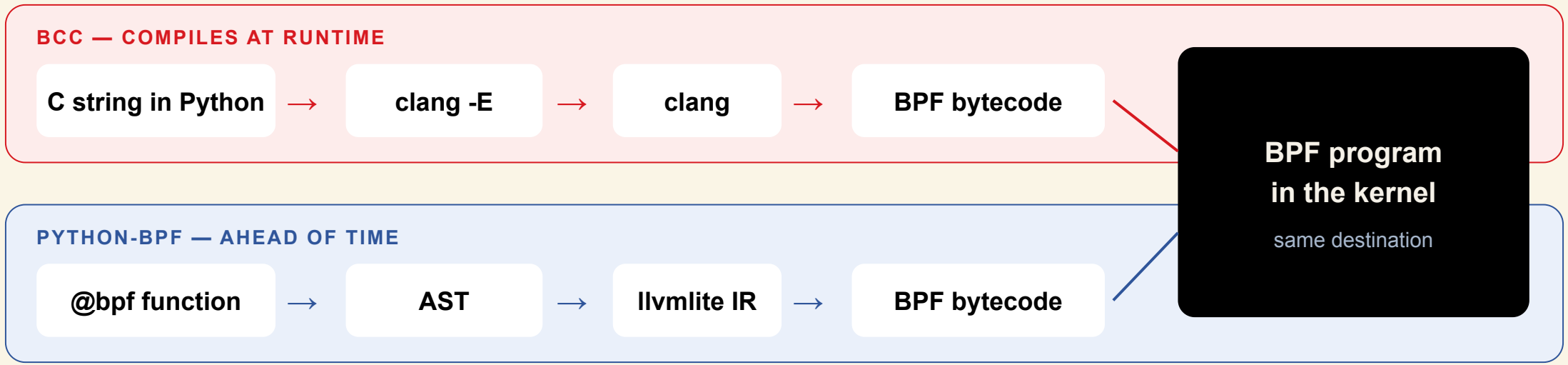
Every number in this deck is CPU time, not wall — slide 23 explains why.



Same tracepoint, same verifier, same maps → **the difference is in the glue**.

Workload: `sys_enter_execve`

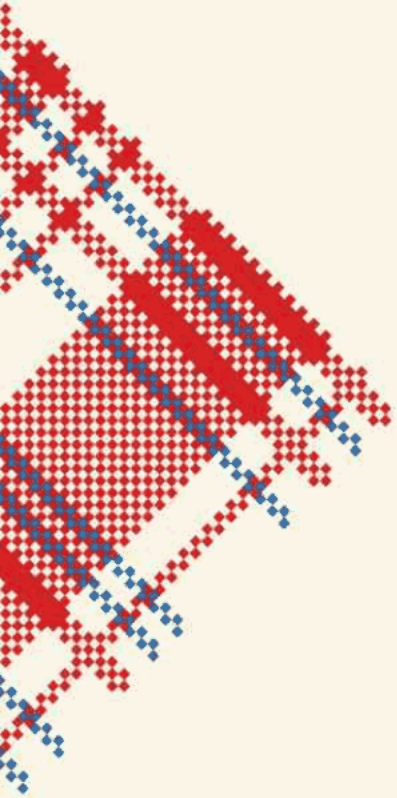
THE ONE DIFFERENCE | how a program becomes bytecode



BCC needs **clang + kernel headers** on the host, every run.

Python-BPF needs **neither** once built.

This is exactly part 1 — startup — which is where we start.



01

Which starts faster?

From process launch to a loaded BPF program.





SETUP | one workload, held constant

ITEM	VALUE
tracepoint	`sys_enter_execve`
trigger	2,000,000 `fork` + `execve` pairs (C binary)
program	`map_1_perf_timed_count.py` — map counter + perf event
host	vmnono (`nono1`), 3 CPUs

Both frameworks run the same program — so any difference is the framework, not the program.

Sequential trigger: fork, wait, fork — about 0.7–0.9 ms per event.

BCC · C inside a Python string28–46

```
import threading
import time
import resource
import signal
from bcc import BPF

main_ident = threading.get_ident()
main_native = threading.get_native_id()
interrupted = False

def handler(signum, frame):
    global interrupted
    interrupted = True

signal.signal(signal.SIGINT, handler)

program="""
BPF_HASH(count_hash, u32, u64);

struct data_t {
    u32 uid;
    u64 count;
    char comm[16];
};

BPF_PERF_OUTPUT(events);
```

Python-BPF · decorated Python36–54

```
import threading
import resource, time, signal
from pythonbpf import BPF, section, bpf, map, bpfglobal, struct
from pythonbpf.maps import HashMap, PerfEventArray
from pythonbpf.helper import uid, comm
from ctypes import c_void_p, c_uint32, c_uint64, c_int64

interrupted = False
def handler(signum, frame):
    global interrupted
    interrupted = True
signal.signal(signal.SIGINT, handler)

@bpf
@struct
class data_t:
    uid: c_uint64
    count: c_uint64
    last_comm: str(16)

@bpf
@map
def Count() -> HashMap:
    return HashMap(key=c_uint32, value=c_uint64, max_entries=1024)

@bpf
```

BCC runtime compile	**0.45 s**	Fork clang, preprocess, compile.
Python-BPF ahead of time	**0.02 s**	Load an object that already exists.

Python-BPF starts about **22× faster**. Ranges: BCC 0.375–0.580 s · Python-BPF 0.020–0.027 s.

Init CPU, 2M count arm, `results/unprofiled`.

AT INIT	BCC	PYTHON-BPF
child process	none	llc
header blocks read	20,768	160

BCC compiles the C **in its own process**. Python-BPF hands compilation to a child and ships a finished object.

> The 0.02 s is not free compile — it is compile that happened **before** the run.

blocks = 512-byte reads at init · `time -v`, 2M count arm.

02

Which costs less per event?

Two million events, same program, CPU only.

count	PRINTS ONE LINE	The realistic tool — this is the arm everything else is measured against.
empty	DOES NOTHING	No field read, no print. It gives us the **floor**: the cost of an event with no Python work.
print	PRINTS A CONSTANT	Same output cost as count, none of the field reads.

One program shape, three bodies — **the kernel program never changes**.

Source: passes 1–3, `results/unprofiled`.

BCC

2,000,000

events delivered — trigger sent 2,000,000

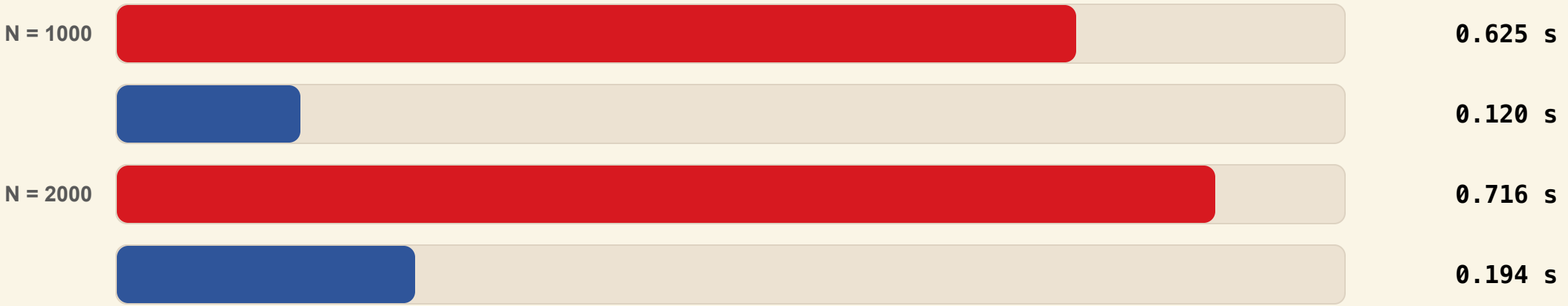
PYTHON-BPF

2,000,000

events delivered — trigger sent 2,000,000

Delivery is equal → **CPU is the only open question**.

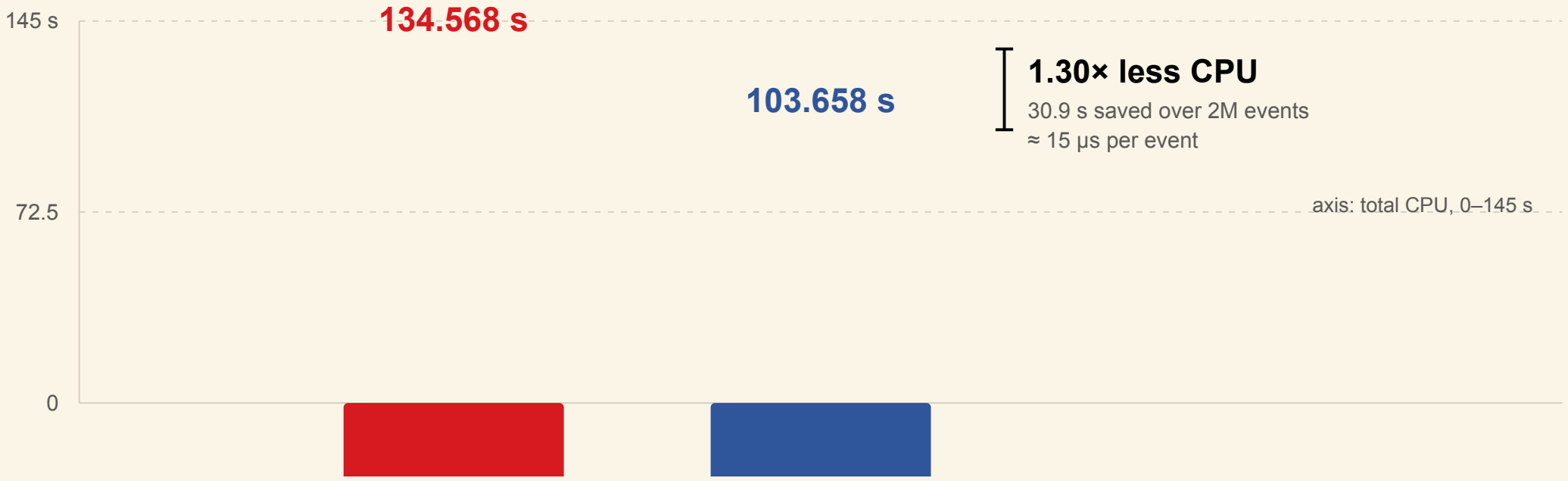
All six 2M arms · log shows 2000001 = summary formula, not an event.



Ratio 5.2× at N=1000, 3.7× at N=2000. Axis: total CPU, 0–145 s (full scale shown on the next slide).

At N ≤ 2000, **startup dominates** — these are not per-event numbers.

Total CPU, count variant, `results/unprofiled`.



Startup is 0.3% of a 1763 s run — **what is left is per-event cost**.

Total CPU, count variant · wall 1763.6 s vs 1768.6 s.

BCC	**103.3 s**	with an empty callback
Python-BPF	**84.0 s**	with an empty callback

The callback does nothing. BCC still spends **19.2 s more** — ratio 1.23.

> So the cost is in **making the call**, not in what the callback does.

`abl_A_empty`, 2M, `results/unprofiled`.



Python-BPF binds with pybind11 · **BCC binds with ctypes** — and Python-BPF pays the ctypes cost on **0%** of cycles.

Share of PMU cycles with that frame anywhere on the stack · the same three names hold in the other arms.

84% of BCC's cycles sit in `ctypes + libffi`. So does libbcc depend on `libffi`?

ASK · PAUSE · THEN PRESS →

No. Not one ffi library in its dependencies.

libffi arrives through `import ctypes`, in the same process, on a different path.

Measured 2026-10-18 · `readelf -d /usr/lib64/libbcc.so.0.36.0 | grep NEEDED`

01 `readelf -d libbcc.so.0.36.0 | grep NEEDED` **— no ffi anywhere**

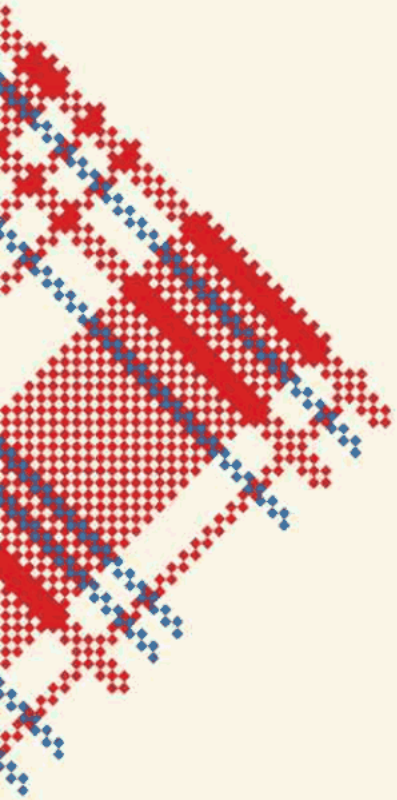
```
libclang-cpp.so.21.1   libLLVM.so.21.1   libelf.so.1    libbpf.so.1
libstdc++.so.6         libm.so.6         libgcc_s.so.1  libc.so.6
```

02 libffi enters on a **separate branch** of the same process

```
python3
 +- bcc/__init__.py:17  import ctypes as ct
 |   +- _ctypes...so -> libffi.so.8      IMPORT cost
 +- bcc/libbcc.py       ctypes.CDLL("libbcc.so.0")
     +- libbcc.so.0  -> libbpf.so.1      eBPF cost
```

Two branches off CPython — **no edge between them**.

Measured 2026-10-18 · libclang-cpp + libLLVM are the larger part of the same trace, not libffi.





03

Who does the waiting?

The read loop — where I had it wrong twice.



`b.perf_buffer_poll(1)` vs `perf.poll(1)`
Are they waiting on the same thing?

ASK · PAUSE · THEN PRESS →

Yes. **Same three perf buffer fds, same 1 ms timeout.**

Only the syscall underneath differs — the next slide shows it.

gettid	**1,907,735** **vs 3**	BCC's callback path pays one per event.
perf buffer wait same 3 fds, same 1 ms timeout	**poll 1,978,114** **epoll_wait 2,037,562**	The wait is **not** eBPF work.
bpf + **perf_event_open**	**19 vs 31**	The real eBPF syscalls — nearly equal.

1.96 syscalls per event for BCC · **1.03** for Python-BPF.

`strace -f -T`, counts only — under strace durations are not usable.

BCC · POLL

1 ms timer → **1.375 ms**

2,037,562 vs 1,978,114 calls

PYTHON-BPF · EPOLL_WAIT

1 ms timer → **1.373 ms**

95% of wakes carry data, not timeouts

Same timer to three digits. The 2.9% total gap is 3% more calls — that arm ran 5 s longer.

epoll earns its keep when you watch **hundreds** of fds. At three, the work is the same.

Means from `strace -T` · timeout 1.375 vs 1.373 ms · with data 0.622 vs 0.675 ms.

wall	**1763.6 s** vs **1768.6 s**	5 s — 0.3%
CPU	**134.6 s** vs **103.7 s**	**30.9 s — 30%**

> The trigger sets the pace, not the framework — so every number in this deck is CPU.

2M count arm · `results/unprofiled`, `[TIMING]`.

pre + gate

2 s / 1 s + 0.5 s

bash, before launch. The gate fires only once the program reports it is polling.

t_init

0.45 s / 0.02 s

The term that separates them — 0.45 s against 0.02 s, the only part the framework controls.

t_poll

1763.6 / 1768.6 s

The loop. It **contains** the trigger — and it is nearly the same in both.

The trigger (~1400 s) runs **inside** the poll loop — not beside it.

`experiment/scripts/rebenc_unprof.sh`, the arm function.

experiment/scripts/rebenc_unprof.sh48–68

```
#!/bin/bash
# sys_enter_execve with_map, UN-PROFILED (no strace, no perf record).
# Variant-agnostic: VAR = count | A | B  (canonical + both ablation variants).
# Same mechanism as rebenc_abl2m.sh; output path passed in via OUT so the three
# passes can land in separate trees.
#
# vmnono (nono1) only. Output NOT under /tmp.
set -u
TAG="${TAG:-$(date -u +%Y%m%dT%H%M%SZ)}"
N="${N:-100}"
VAR="${VAR:-count}"
OUT="${OUT:-/root/experiment/results/unprofiled}"
GAP="${GAP:-60}"
T="${T:-0.5}"
mkdir -p "$OUT"

[ "$(hostname)" = nono1 ] || { echo "[HALT] vmnono/nono1 only"; exit 1; }

case "$VAR" in
  count) SUFFIX="map1_count" ;;
  A)     SUFFIX="ablA_empty" ;;
  B)     SUFFIX="ablB_print_const" ;;
  *) echo "[HALT] VAR must be count|A|B"; exit 2 ;;
esac

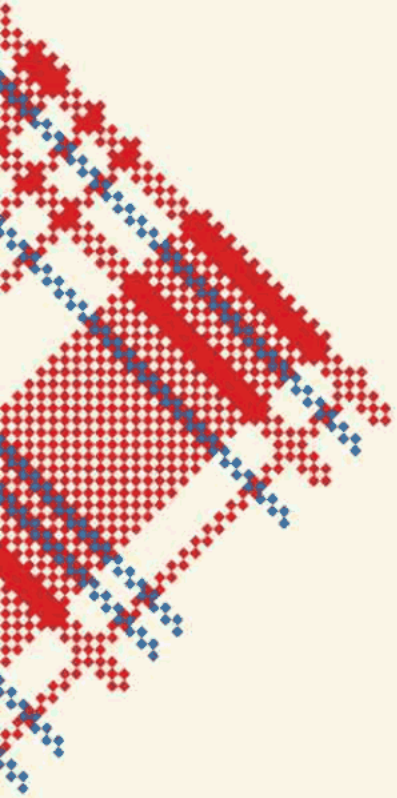
PY_BCC=/usr/bin/python3
```

↑ ↓ move · click to pin · scroll for the rest of the file

04

What do you lose?

Four limits in Python-BPF today, all found by building a real tool.







	Python-BPF	BCC
raise the signal	user space, after `trace_pipe`	`bpf_send_signal`, in kernel
helpers available	18	36
`bpf_send_signal`	absent	present

Both kill the trigger. BCC needs no user space hop — trigger returns `rc = −9`.

> But **neither denies** a syscall: the signal fires on arrival back from the probe. A seccomp filter runs before the call.

Measured on vmnono, BCC 0.36.0 · helper counts from `pythonbpf/helper/`.

BCC · signal raised in the kernel program51–74

```
#!/usr/bin/env python3
# GhostLock enforcer half, BCC version.
#
# This is the A/B partner of ghostlock_enforce.py (Python-BPF). Same job: watc
# kprobe/do_futex, tag the GhostLock op set, and act on FUTEX_CMP_REQUEUE_PI.
#
# The difference is the helper surface. BCC ships bpf_send_signal, so the sign
# is raised FROM THE KERNEL PROGRAM rather than read back through trace_pipe.
# Python-BPF has no such helper, so ghostlock_enforce.py has to print the line
# read it in userspace, and os.kill() from there.
#
# What is STILL true in this version, and worth saying on the slide:
#   * bpf_send_signal delivers SIGKILL on return from the tracepoint, which is
#     AFTER do_futex has run. So the syscall that matters has already executed
#     Being in the kernel removes the trace_pipe hop, not the ordering.
#   * No in-kernel deny. Neither library can say "return an error here instead
#     XDP drop is a verdict; a kprobe is not. A seccomp filter acts before the
#     syscall, which is why it can deny where a signal cannot.
#
# Probe point: kprobe/do_futex, matching the Python-BPF tracker. The sys_*
# wrapper needs a two-step deref for the op; do_futex takes it as a plain arg.
from bcc import BPF

program = r"""
// Two headers are deliberately NOT included, and both cost a debugging round:
//   linux/futex.h -> drags in mutex code clang cannot emit for BPF
```

Python-BPF · print, read trace_pipe, os.kill68–91

```
#!/root/experiment/.venv/bin/python3
# GhostLock enforcer half. Python-BPF has no in-kernel deny primitive (no BPF-
# section, no bpf_override_return helper), so the block is a userspace policy
# action fed by the PBPF signal: kill the task seen issuing FUTEX_CMP_REQUEUE_
# (tag=3). The kernel-side equivalent is the seccomp PI-futex filter from
# Atlas/Dots/Things/CVE-2026-43499 GhostLock/GhostLock - Mitigation.md.
#
# Reads /sys/kernel/tracing/trace_pipe (consuming reader): each printk line fr
# the PBPF program is delivered exactly once.
import os
import re
import signal
import sys
import time
from pythonbpf import bpf, bpfglobal, section, BPF
from pythonbpf.helper import pid
from vmlinux import struct_pt_regs
from ctypes import c_int64

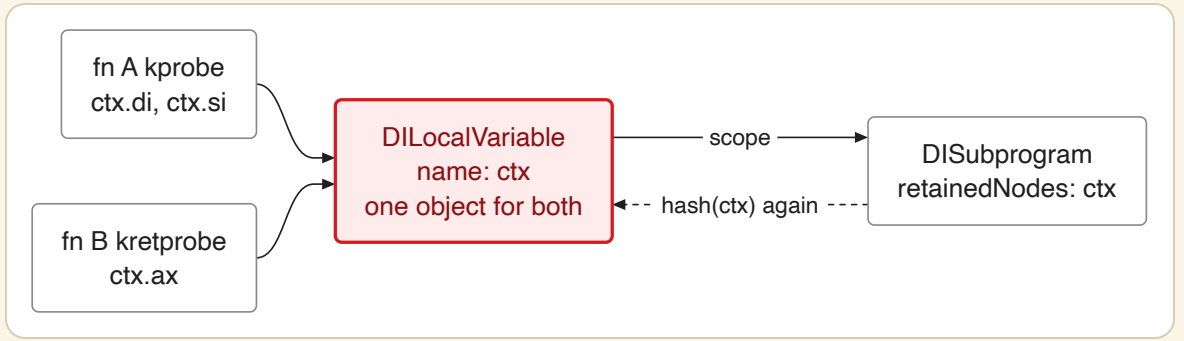
# A pid first seen at tag=3 is acted on once. It stays suppressed only while t
# task that owns it is still alive. Pids are recycled, so the entry must expir
# or a kill of pid N hides a later task that is handed pid N.
REAP_INTERVAL = 2.0

@bpf
@bpfglobal
```

Two sections annotate `ctx: struct_pt_regs`. The annotations are equal. llvmlite caches metadata by value. Both functions then get one `DILocalVariable` node, and Python-BPF adds `scope` to it.

DEBUG NODE	SHARED BY CACHE	LOOP
`DIDerivedType`	yes	no
`DISubroutineType`	yes	no
`DILocalVariable`	yes	yes
`DISubprogram`	no, name differs	holds the loop

Entry reads `ctx.di` and `ctx.si`. Exit reads `ctx.ax`. Both sections annotate the same type. Equal type, equal node.



`hash(ctx)` enters `DISubprogram` and comes back to `ctx`. The loop has no end, so Python raises `RecursionError`. Fix: build one node for each function. Until then: **one section per file**.

Measured on vmnono · Python-BPF 0.1.9, 2026-11-06 · the union warnings are not the cause.

Program: `kprobe_unlink.py` — `@section("kprobe/do_unlinkat")`, a kprobe on `do_unlinkat`: `ctx.di` is `dfd`, `ctx.si` is arg 2, a `struct filename *`. Three attempts to read its name:

ATTEMPT	HELPER	RESULT
read `struct filename.name`	`probe_read`	Struct type … not found
read into local `str(256)`	`probe_read_str`	Variable 'buf' not found
read into `c_uint64`	`probe_read_str`	8 bytes — truncates

Not garbage: the **second dereference** is missing. `probe_read` gives an `i64`; `probe_read_str` wants an `i8*`. No `inttoptr`.

KPROBE_UNLINK.PY · ONLY VERSION THAT COMPILES

```
@bpf
@section("kprobe/do_unlinkat")
def do_unlinkat_entry(ctx: struct_pt_regs) -> c_int64:
    fname_ptr = ctx.si          # struct filename *, arg 2
    name_str  = 0               # 8-byte stack slot
    probe_read(name_str, 8, fname_ptr)
    print(f"name={name_str}")   # 0xffff8f2d4705c000
    return 0
```

Reads the `name` pointer as a signed `i64` — the string itself is never dereferenced.

CONSTRUCT	WHAT HAPPENS
`ctx.args[0]`	falls through — no code emitted
`count += 1`	a plain add, not atomic

A map counter **can lose updates** — BCC emits one atomic read-modify-write for the same line. That is the `Count.update(...)` line from the program you saw in the setup.

Both are compiler gaps, not design limits.

`expr_pass.py:730` (subscript) · `expr_pass.py:227` (add).

One `print`, four values → silent. Is that Python-BPF, or the kernel?

ASK · PAUSE · THEN PRESS →

The kernel. **Both frameworks. `bpf_trace_printk` takes three values after the format string.**

fmt + size + 3 varargs = 5, the eBPF helper argument cap. A framework that tried four would fail the verifier.

The count is not the finding. **The silence is.**

RESULT | 2M events, count variant

AXIS	BCC	Python-BPF	WHY
startup CPU	0.45 s	0.02 s	BCC compiles C at runtime
per-event CPU	134.6 s	103.7 s	ctypes bridge vs pybind11 bridge
syscalls per event	1.96	1.03	one gettid per event in BCC
peak RSS	175 MB	22 MB	clang loaded in the BCC process

Python-BPF wins all four axes. BCC wins one thing today — next slide.

Python-BPF RSS is the parent process; its llc child peaks at 85 MB, separately.

RESULT | pick by job, not by speed

PICK PYTHON-BPF

startup **0.02 s**
CPU at 2M **1.30× lower**
syscalls **1.03 / event**
memory **22 MB**

PICK BCC

a struct as a map value **works today**
plus in-kernel signal and BPF-LSM deny

Boundary: a kprobe has **no verdict**. `bpf_send_signal` queues a signal; it fires after the syscall returns. Deny needs a pre-call hook: seccomp, or BPF-LSM — `attach_lsm` exists in BCC, Python-BPF has none.

October numbers only · both frameworks run the same program.

01	**Starts faster?** Python-BPF · 0.02 s vs 0.45 s	the work moved earlier
02	**Less CPU per event?** Python-BPF · 103.7 s vs 134.6 s at 2M	one bridge is cheaper
03	**Who waits?** both — same 1 ms timer, same fds	not the story
04	**What do you lose?** deny · sections · kernel strings · atomics	this decides it

The kernel is the same. **The glue is the difference.**

Click an answer to jump back to its section.

PYCON HK 2026

Q&A

Slides and data → **github.com/nizarakbarm/pycon-hk-2026-bcc-vs-pythonbpf**

Python-BPF github.com/python-bpf/python-bpf · BCC github.com/iovisor/bcc